

```

// BACKWARDS DIJKSTRA
DijkstraResult backwards_dijkstra(const Graph& g, int s, int t) {
    // stop conditions
    if (s < 0 || s >= g.get_nr_vertices())
        throw std::out_of_range("Start vertex out of bounds");
    if (t < 0 || t >= g.get_nr_vertices())
        throw std::out_of_range("Target vertex out of bounds");

    DijkstraResult result;

    // min-heap priority queue
    std::priority_queue<
        std::pair<int, int>,
        std::vector<std::pair<int, int>>,
        std::greater<std::pair<int, int>>> // flips the default priority queue
    which is usually max-heap
    > q;

    q.push({0,t}); // start from target
    result.dist[t] = 0; // distance from t to itself is 0
    bool found=false; // optimization (stop when we reach s)

    while (!q.empty() && !found) { // queue still has vertices + haven't
reached s yet
        // extract best node
        int x = q.top().second; // current vertex
        int current_dist = q.top().first; // best known distance to current
vertex
        q.pop();

        if (current_dist != result.dist[x]) // only process best distance for
each vertex
            continue;

        for (auto const& [y,edge]:g.parse_in(x)) { // traverse inbound edges
(y->x)
            if (result.dist.find(y)==result.dist.end() ||
result.dist[x]+edge->cost < result.dist[y]) { // relaxation step
                result.dist[y]=result.dist[x]+edge->cost; // update
                q.push({result.dist[y],y}); // add updated y to queue
                result.next[y]=x; // path (from y, the next step towards t is
x)
            }
        }

        if (x==s)
            found = true;
    }
    return result;
}

```

```

// graph - read only
// s - start vertex
// t - target vertex
std::vector<int> lowest_cost_walk(const DijkstraResult& result, int s, int t)
{ // returns a list of vertices (the path)
    //DijkstraResult result = backwards_dijkstra(g, s, t); // run dijkstra
    // in result.dist -> costs to reach t
    // in result.next -> how to move towards t
    if (result.dist.find(s)==result.dist.end()) // no path from s to t
        return {}; // return the empty vector
    std::vector<int> path; // the result path (vector) built from the
beginning
    int current = s;
    path.push_back(current); // s (start) is the first element - add vertex to
the path
    while (current != t) { // loop until reached t (target)
        current=result.next.at(current); // go to the next vertex on the
optimal path (next[0]=2, next[2]=5 => 0-2-5)
        path.push_back(current); // add vertex to the path
    }
    return path;
}

```